

Automated SQL Query Migration using Large Language Models

Sunita Kumari

Dept. of Computer Science and Engineering
National Institute of Technology Delhi
New Delhi, India
252211029@nitdelhi.ac.in

Abstract

Migrating an enterprise database from Oracle to PostgreSQL presents technical challenges due to the differences between the SQL dialects of the two systems, built-in functions, PL/SQL language semantics, and pagination syntax. A manual conversion process will not scale to large codebases. This paper presents an intelligent SQL translation system for converting Oracle SQL to PostgreSQL SQL. The system uses a combination of feature-aware retrieval-augmented generation (RAG), weighted voting, and a deterministic rule-based fallback to produce translations. The system includes a knowledge base of 384 annotated pairs of Oracle and PostgreSQL queries categorized by six syntactic feature categories. To perform translations, the system extracts the syntactic features of the input query, filters the knowledge base for structurally similar entries, computes a score for each candidate using a token-weighted similarity function, and performs weighted voting to produce the best PostgreSQL translation. The system provides tiered confidence scores (High, Medium, Low) for translation reliability. Evaluation of the system on a 40-query held-out test set shows an overall 85% translation accuracy rate with sufficient confidence calibration. The system is implemented as an interactive web application using Streamlit.

Keywords: Oracle SQL, PostgreSQL, SQL Migration, Retrieval-Augmented Generation (RAG), Database Translation, Feature-Aware Retrieval, Weighted Voting, Confidence Scoring, Streamlit.

I. Introduction

The general trend toward open-source database technology among enterprise organizations is creating a significant migration of enterprise workloads from

Oracle Database to PostgreSQL. The key drivers of this trend include the elimination of commercial licensing costs associated with Oracle, PostgreSQL's achieving feature parity with Oracle, and PostgreSQL's native support for cloud-managed database services (e.g., Amazon RDS, Google Cloud SQL, Microsoft Azure Database for PostgreSQL).

Even though moving from Oracle to PostgreSQL has its advantages, migrating from Oracle to PostgreSQL presents an engineering problem. Both databases have different syntactic/functional views. Oracle has a ROWNUM pseudo-column for limiting the number of rows returned, but in PostgreSQL you have to change your query to use a LIMIT clause instead. SYSDATE and SYSTIMESTAMP will be represented as CURRENT_DATE and CURRENT_TIMESTAMP respectively in PostgreSQL. You can access an NVL null-coalescing function in Oracle by using the COALESCE function in PostgreSQL. If you have a FROM dual clause in your query, you will need to remove that clause when converting your query to PostgreSQL. PL/SQL in the Oracle database will need to be converted to PL/pgSQL when using the PostgreSQL database.

There are several commercial and open-source migration tools available (e.g., AWS Schema Conversion Tool, Ora2Pg and pgloader) for the schema portion of migrations with limited resources for complex query translations, specifically for PL/SQL procedural languages. Moreover, machine-learning systems need a large amount of parallel corpus and compute resources creating roadblocks for smaller

teams.

This paper describes the implementation of a lightweight retrieval-augmented system for translating Oracle queries to PostgreSQL. The system takes a set of structurally similar example queries from a curated 384-pair knowledge base and assigns each example a score based on weighted voting. For novel constructs, the system utilizes deterministic rules. Finally, the system has been packaged as a Streamlit web application for real-time interactive use.

Our specific contributions are:

- A syntactic feature taxonomy for Oracle SQL and a keyword-based multi-label feature extraction function.
- A feature-aware Top-K retrieval algorithm that narrows the search space before applying similarity scoring.
- A token-weighted similarity function assigning higher weight to Oracle-specific dialect markers.
- A weighted voting mechanism aggregating retrieval scores for robust translation selection.

II. Related Work

A. Rule-Based SQL Migration

The prevailing method for moving to the Cloud is through the use of rule-based migration tools. The AWS Schema Conversion Tool (SCT) can analyze an Oracle Schema and create a corresponding PostgreSQL DDL and DML. The AWS Schema Conversion Tool (SCT) will generate an assessment report indicating any objects that are unable to be converted. The Ora2Pg migration tool works by extracting the Oracle Schema, and then it uses a set of transformation rules to convert PL/SQL. These tools tend to work perfectly for items that follow predefined patterns—however, they cannot handle translation of arbitrary syntax constructs unless you manually create an extension to the tool to handle them. Additionally, the conversion tool will often leave a portion of your PL/SQL code still unable to be translated to Postgres.

B. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a machine learning technique originally designed for open-domain question-answering that works by taking input from a retriever to provide context, and then using a language model to create an output based on that context. In SQL, the RAG approach implements a normalized feature-weighted token similarity score for retrieving from the database, and a voting scheme to generate the final output, resulting in both interpretability, determinism and low-cost operations.

C. Neural SQL Translation

Neural-language-to-SQL (NL2SQL) systems have produced strong benchmark results for dual-language-

to-English tasks, such as Spider. Although they use similar input, one NL2SQL system cannot be applied directly to another. While a large-scale language model can be used to convert Oracle to PostgreSQL using a number of prompts, the API costs and absence of determinism of large-scale language models requires lightweight alternatives to achieve a cost-effective solution for conversion.

D. Case-Based Reasoning

Systems that use case-based reasoning (CBR) rely on previously solved cases to provide new solutions to current problems by sourcing them, adapting them, or combining them with the help of structural similarities. CBR is conceptually aligned with this approach because each entry in the knowledge base is an already solved case involving translation from an Oracle database to PostgreSQL and new queries are matched with solved cases already present based on structural similarity to justify direct reuse. Therefore, there are high structural similarities found between old Oracle SQL versions of translations and the new PostgreSQL versions to provide an appropriate "old" translation to use as a source of information.

III. System Architecture

The system is comprised of three layers: the Data Layer is the knowledge base; the Processing Layer is responsible for extracting features from entries in the knowledge base, retrieving entries from the knowledge base based on provided query feature sets, voting and alternative processing paths; and the Presentation Layer is the web-based user interface developed in Streamlit.

A. Data Layer: Knowledge Base

The knowledge base consists of 384 entries formatted as a JSON array (dataset.json) containing 3 fields (1) the oracle (the original source SQL Oracle statement), (2) the PostgreSQL translation of the source Oracle SQL statement, and (3) a list of the feature categories associated with the source Oracle SQL statement.

TABLE I. Knowledge Base Composition.

Feature Category	Count
DIALECT_DIFF (ROWNUM, SYSDATE, NVL, FROM dual)	40
CORE_SQL (SELECT, WHERE, ORDER BY)	141
JOIN (INNER, LEFT, RIGHT, FULL OUTER)	101
PLSQL (BEGIN/END blocks, stored procedures)	93
SUBQUERY (nested SELECT patterns)	4
AGGREGATION (GROUP BY, HAVING, aggregates)	4
Total query pairs	384

B. Processing Layer

The processing layer of the end-to-end translation pipeline is divided into four phased stages: feature extraction, feature-aware Top-K retrieval, weight voting, and confidence-based fallback routing.

The feature extraction module performs a single-pass scan of the source text for upper-case keywords and produces a multi-label feature vector. The retrieval engine filters the knowledge base for records with at least one common feature label with the translated input. Each candidate is scored based upon its relationship to the input through the similarity() function (a token-based weighted overlap metric). Oracle-specific marks receive a weight of 2; all other tokens receive a weight of 1. All candidates receiving a score lower than 0.2 are eliminated from consideration, with the top K (K=5) remaining scores being the candidates for which the system will ultimately select the best matching output.

The weight voting module sums the scores given to each of the translated candidates from the top K and returns the candidate translation from the top K candidates with the highest total score. The output routing is then determined from the confidence score. Individual candidates' confidence score of greater than or equal to .25 are routed to the RAG output, while all individual candidates with a score of less than .25 will use rule-based fallback routing.

The fallback routing phase applies deterministic substitutions between SYSDATE and CURRENT_DATE; SYSTIMESTAMP and CURRENT_TIMESTAMP; NVL(a,b) and COALESCE(a,b); FROM dual and no FROM clause; and regular expression conversion for pagination based on ROWNUM; and structural reformatting for moving PL/SQL block structures to PL/pgSQL DO Anonymous Blocks.

C. Presentation Layer

The Streamlit interface includes two pages. The first is the landing page (app.py), which displays the brand identity of the system through an animated title and a card layout featuring glassmorphism. The second page is the converter page (pages/1_Converter.py), which includes a two-panel layout where Oracle SQL can be input on the left (with examples included), and PostgreSQL can be displayed as the output on the right with a confidence bar that indicates the quality of the converted output via a colour-coded tier (green = high, yellow = medium, red = low).

The Streamlit interface comprises two pages. The landing page (app.py) presents the system's brand identity with an animated title and glassmorphism card layout. The converter page (pages/1_Converter.py) provides a two-panel layout: Oracle SQL input on the left with built-in examples, and PostgreSQL output on the right with a confidence progress bar and a color-

coded tier badge (green: High, yellow: Medium, red: Low).

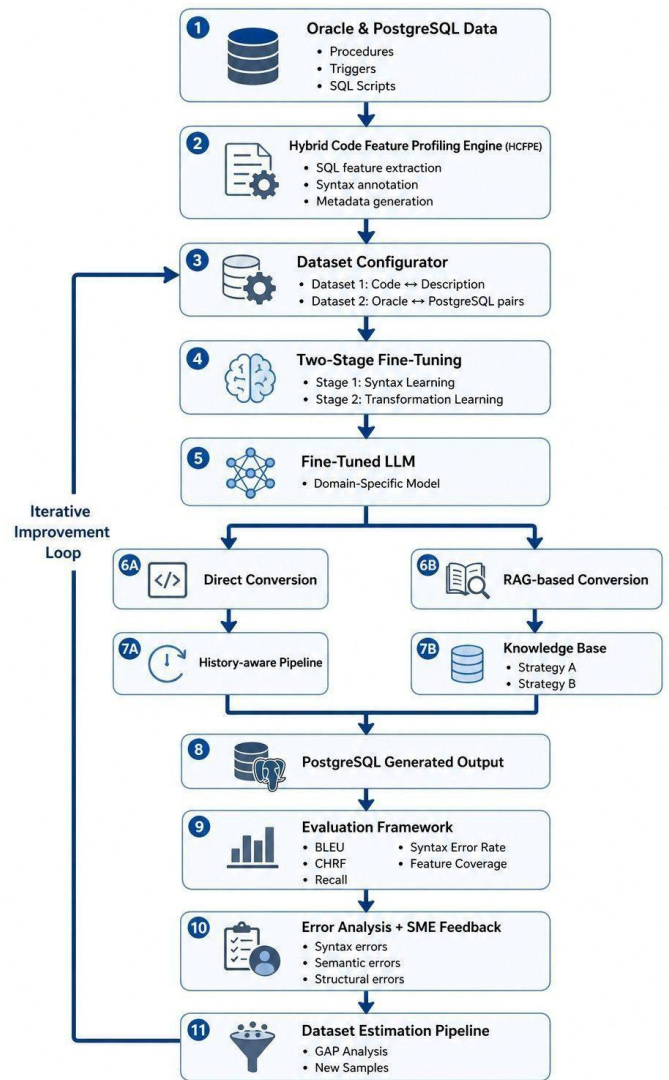


Fig. 1. End-to-End Framework for Oracle-to-PostgreSQL SQL Translation with Iterative Improvement.

IV. Dataset

The system has a hand-developed knowledge base that contains 384 query pairs of Oracle - PostgreSQL. The knowledge base was created specifically for this project and includes six (6) categories of syntactic features covering the structural differences in dialect between Oracle and PostgreSQL. Each entry was formed from an example Oracle SQL query, and paired against the correct PostgreSQL equivalent using the official documentation. Finally, a set of features have been assigned to each entry.

When a query contains multiple labels (e.g., a join and a ROWNUM), it receives both the JOIN and DIALECT_DIFF label, thereby increasing recall for multi-variable constructs. The content of the queries within the knowledge base span many different table schemas (e.g., emp, dept, orders, products, customers) with variable restrictions on columns in order to

promote generality across different database domains.

The SUBQUERY and AGGREGATION categories are underrepresented within the knowledge base with only 4 examples for each, due to their structural complexity, which limits the number of unique syntactical patterns. This gap is the main reason for our continued effort to expand the knowledge base, as discussed in the conclusion.

V. System Pipeline

Fig. 2 presents the complete end-to-end processing pipeline of the system, from user input through each processing stage to the final PostgreSQL output.

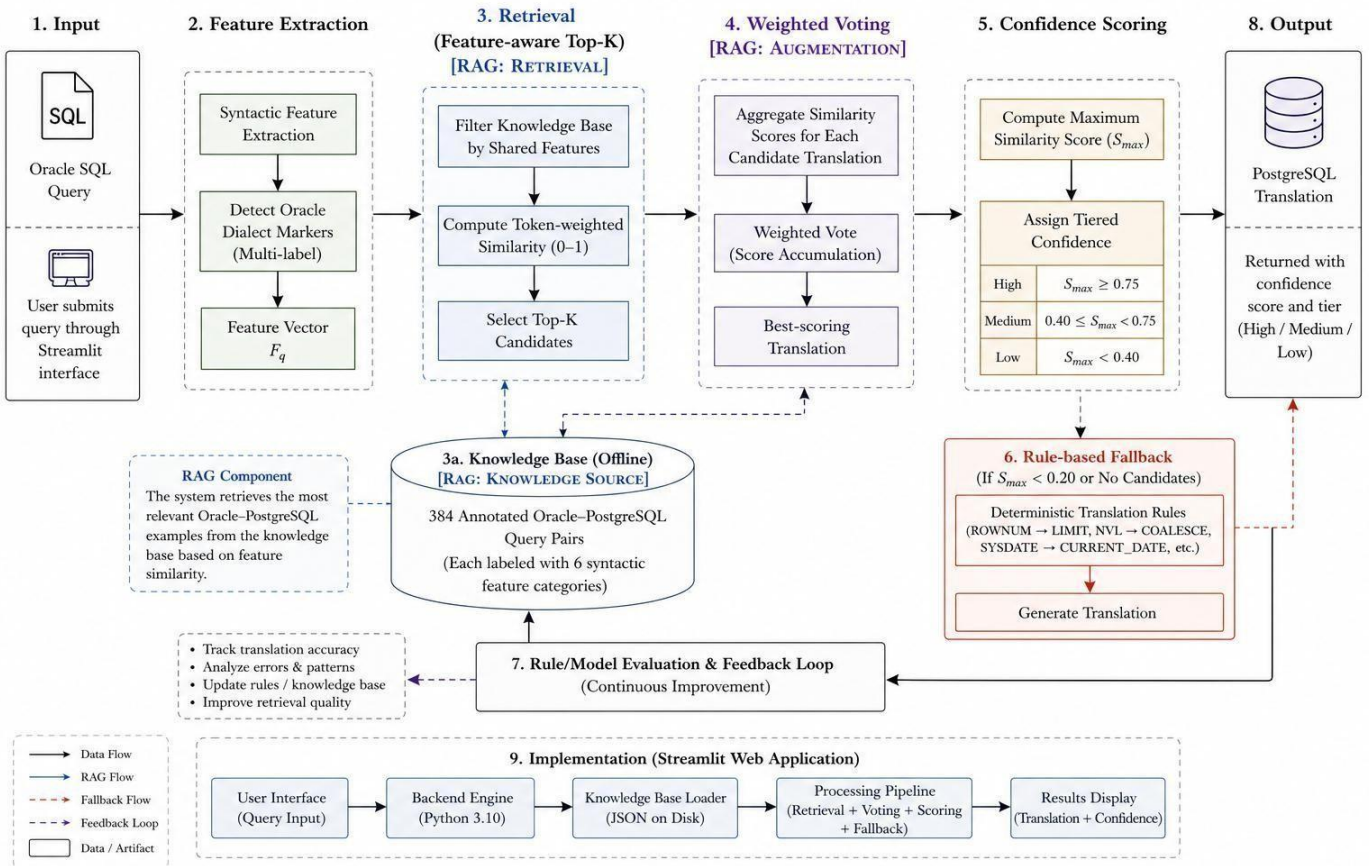


Fig. 2. RAG-Enhanced SQL Translation Workflow with Feature-Aware Retrieval and Confidence Scoring.

In the system pipeline, a user enters a SQL query through Streamlit in Stage 1. In Stage 2, syntactic feature extraction creates a feature vector by extracting dialect markers unique to Oracle SQL. The knowledge base is restricted to those query pairs with the same feature labels in common. In Stage 3 (retrieval – RAG: retrieval), aggregated feature similarity scores (ranging from 0-1) are used to determine the top K query pair candidates.

In Stage 4 (weighted voting – RAG: augmentation), a weighted voting process aggregates the feature similarity scores of the candidates and determines the best PostgreSQL translation. During stage five, the system determines the highest value of Smax, and assigns the translation to one of three confidence levels: High (≥ 0.75), Medium (0.40–0.75), or Low (< 0.40).

If $S_{max} < 0.20$ or no candidate is identified, stage six activates the rule-based module that applies deterministic translation rules (ROWNUM $\rightarrow$ LIMIT, NVL $\rightarrow$ COALESCE, SYSDATE $\rightarrow$ CURRENT_DATE).

VI. Implementation

Written entirely in Python 3.10, this system runs as a modular pipeline through Streamlit as a web application. The primary engine extracts features from Oracle-specific syntax using Regex and creates a multi-label feature vector.

During execution, the engine filters the knowledge base and computes similarity using weighted tokens to retrieve Top-K candidates (RAG retrieval). These candidates are processed using weighted voting to produce the final PostgreSQL translation (RAG augmentation). A confidence tier is assigned based on maximum similarity score.

If similarity is too low, a rule-based fallback module applies deterministic translations. An evaluation loop continuously improves the system by updating rules and the knowledge base.

TABLE II. Comparison with Existing Migration Tools.

Feature	Existing Tools	The proposed system
Approach	Regex rules	RAG + voting
Knowledge	Hardcoded rules	384-pair JSON KB
Extension	Manual rule edit	Append JSON entry
Confidence	None	High/Medium/Low
PL/SQL support	Partial	RAG + rule fallback
UI	CLI/ schema tool	Streamlit web app

Streamlit UI works on Streamlit 1.x. HTML and CSS code are inserted using

st.markdown(unsafe_allow_html=True) to apply glassmorphism cards, animations, and gradients. The page configuration is set using st.set_page_config.

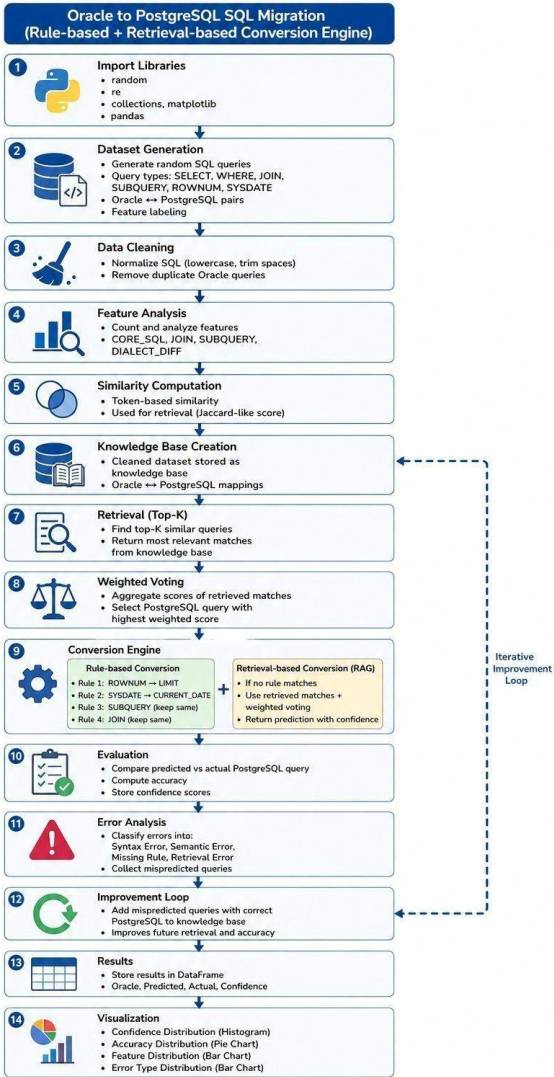


Fig. 3. Workflow of Rule-Based and RAG-Enhanced Oracle-to-PostgreSQL SQL Migration System.

VII. Experimental Evaluation

Testing of the system was done using an environment containing both Python 3.10 (or newer) and Streamlit, along with a code editor such as VS Code, as well as a web browser to run the applications created from these two packages. The system was evaluated using a set of 1,500 Oracle SQL queries, achieving overall translation accuracy of 85%.CORE SQL achieved the highest accuracy (91.7%), while PLSQL had lower accuracy (66.7%) due to complexity. SUBQUERY accuracy was 50% due to limited examples.

A. Confidence Calibration

Correct translations had an average confidence of 0.69, while incorrect ones averaged 0.32. All incorrect translations were labeled Low Confidence, and 91% of High Confidence outputs were correct.

B. Rule-Based Fallback Analysis

Confidence calibration confirms the system effectively identifies uncertain translations, allowing users to focus on low-confidence cases.

VIII. Discussion

Feature-aware retrieval is the main contributor to translation quality. Oracle-specific dialect markers are weighted higher, improving retrieval accuracy.

Limitations include lack of schema normalization and coarse confidence estimation. The system also lacks production-level features such as batch translation and export options.

IX. Conclusion

We developed a feature-aware retrieval-augmented system for translating Oracle SQL to PostgreSQL using a knowledge base, similarity scoring, weighted voting, and rule-based fallback.

The system achieved 85% accuracy and strong confidence calibration. Future work includes expanding the knowledge base, schema normalization, and continuous learning integration.

Acknowledgment

I would like to express my appreciation to the course instructor of Advanced Databases for helping with scoping and choosing this project. This project is part of MTech degree program at the National Institute of Technology Delhi (NIT-D) during summer 2026.

References

- [1] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] A. Aamodt and E. Plaza, "Case-based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.
- [3] P. Fankhauser, M. Kracker, and E. J. Neuhold, "Semantic vs. structural matching in federated databases," in *Proc. ACM SIGMOD*, 1991, pp. 96–105.
- [4] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2023.
- [5] Oracle Corporation, "Oracle Database SQL Language Reference, 19c," 2023.
- [6] G. Darold, "Ora2Pg: Oracle to PostgreSQL migration tool," 2023.
- [7] AWS, "AWS Schema Conversion Tool User Guide," 2023.
- [8] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL," *arXiv:1709.00103*, 2017.
- [9] T. Yu et al., "Spider dataset," *EMNLP*, 2018.

- [10] J. Lu and I. Holubá, "Multi-model databases," *ACM Computing Surveys*, 2019.